

GABARITO da 2ª Prova de Linguagens de Programação

Observação

- A prova deve apenas uma parte e o total (6,0) será multiplicado por 1,67 para estabelecer a nota final (com valor máximo 10).

1ª Questão (1,0 pontos): Expressões e Sentenças

Considere o código Javascript a seguir que utiliza o operador ternário ?:.

```
res = (num != 0) && (sum/num >= 7)? "aprovados" : "recuperacao";
```

a. Explique como as avaliações curto-circuito influenciam a corretude do código.

- **RESPOSTA ESPERADA/SUGERIDA:** a avaliação em questão que determina o valor final é (sum/num >= 7), mas só pode ser avaliado se num é diferente de zero. O código pretende considerar essa situação e evitar erros de divisão por zero, além de estabelecer um resultado claro neste caso ("recuperacao").

b. Escreva um código equivalente em uma linguagem qualquer (mesmo hipotética) que **não** permita avaliação com curto-circuito e possua o mesmo comportamento do código anterior.

- **RESPOSTA ESPERADA/SUGERIDA**

```
res = "recuperacao"
if (num != 0) {
    if (sum/num >= 7) {
        res = "aprovados"
    }
}
```

2ª Questão (2,0 pontos): Subprogramas

2.1 - (0,5 pts) Definição e Flexibilidade em Subprogramas

Explique a diferença em termos do **desempenho** da invocação de parâmetros utilizando parâmetros **posicionais** e **nomeados**. Se necessário, contextualize a sua resposta utilizando uma linguagem de programação.

- **RESPOSTA ESPERADA/SUGERIDA**

- Se a passagem de parâmetros é posicional, então em tempo de compilação - para linguagens compiladas e estaticamente tipadas - é possível informar em qual posição relativa de memória está cada um dos parâmetros, o que torna o código gerado rápido em relação à resolução dos parâmetros. Na passagem nomeada, que é típica em linguagens dinâmicas, cada nome de parâmetro precisa ser resolvido em tempo de execução com uma posição de memória, o que tende a tornar o código gerado mais lento em relação à outra passagem.

2.2 - (0,5 pts) Generalidade em Subprogramas

Considere uma implementação de uma versão genérica do Bubble Sort, que seja capaz de ordenar um vetor **vet** homogêneo, composto de dados de tipo não conhecidos a priori. A assinatura da função em C é a seguinte:

```
void bubble_sort_generico (void *vet, int tamanho, int tamTipo, int (*funcaoEMenor) (void *, void *))
```

onde **tamanho**, **tamTipo** e **funcaoEMenor** são, respectivamente, o número de elementos do vetor, o tamanho do tipo armazenado no vetor e uma função que compara dois elementos do vetor. Explique sucintamente por que **bubble_sort_generico** **NÃO** pode trocar dois elementos na posição **j** e **j+1** utilizando as instruções do código abaixo.

```
1 temp = vet[j];
2 vet[j] = vet[j + 1];
3 vet[j + 1] = temp;
```

- **RESPOSTA ESPERADA/SUGERIDA:** Essa troca não pode ser como indicada no código porque em tempo de compilação não é possível dizer qual é o tamanho do dado (por isso uso do * void). Por isso, não é possível dizer qual o número de bytes que devem ser copiados em cada atribuição. O código correto, precisa copiar em memória o número de bytes informado em **tamTipo**, em cada atribuição que for necessária.

2.3 - (0,5 pts) Verificação e Subprogramas

Explique por que em C, ao declarar um subprograma que recebe uma matriz **bidimensional** como parâmetro é necessário indicar uma das dimensões da matriz, enquanto que em Java isso não é necessário.

- **RESPOSTA ESPERADA/SUGERIDA:** É necessário informar uma das dimensões, pois o compilador precisa de uma dimensão para calcular o tamanho dos saltos em memória que devem corresponder a cada coluna da matriz em C. Em Java, isso não é necessário porque a matriz é disposta de outra maneira: como um vetor de vetores e cada um dos vetores possui um atributo que indica qual é o seu tamanho (respectiva dimensão da matriz), que é acessado pela invocação do atributo `length` da matriz/vetor.

2.4 - (0,5 pts) Implementação de Subprogramas

Considere as seguintes implementações de funções `fib` e `fib_R` para calcular o enésimo número de Fibonacci:

```
1  int fib(int n) {
2      int t1 = 0;
3      int t2 = 1;
4      for (int i = 0; i < n; i++) {
5          int prox = t1 + t2;
6          t1 = t2;
7          t2 = prox;
8      }
9      return t1;
10 }
11
12 int fib_R(int n) {
13     if(n == 0) {
14         return 0;
15     }
16     else {
17         if(n == 1) {
18             return 1;
19         }
20         else {
21             return fib_R(n - 1) + fib_R(n - 2);
22         }
23     }
24 }
```

- a) Descreva as operações na pilha de execução do programa para as chamadas `fib(3)` e `fib_R(3)`, em termos de chamadas `empilha(fib(k))` e `desempilha(fib(k))`, onde `k` é o valor do parâmetro passado para a respectiva chamada para a função `fib`. Coloque uma chamada por linha da sua resposta.

- **RESPOSTA ESPERADA/SUGERIDA**

- `fib(3)`

```
empilha(fib(3))
desempilha(fib(3))
```

- `fib_R(3)`

```
empilha(fib_R(3))
  empilha(fib_R(2))
    empilha(fib_R(1))
      desempilha(fib_R(1))
        empilha(fib_R(0))
          desempilha(fib_R(0))
        desempilha(fib_R(2))
      empilha(fib_R(1))
        desempilha(fib_R(1))
      desempilha(fib_R(3))
```

- b) Considerando que um inteiro ocupa 4 bytes nessa linguagem, e considerando **APENAS** o espaço ocupado pelo parâmetro e variáveis locais na AR da função, qual é o maior número de bytes que a execução de `fib(3)` e `fib_R(3)` ocupam na memória? Coloque as respostas para cada uma das chamadas.

- **RESPOSTA ESPERADA/SUGERIDA**

- `fib(3)`: cada invocação 5*4 bytes, então um máximo de 20 bytes.
- `fib_R(3)`: cada invocação 1*4 bytes, então um máximo de 12 bytes, quando na pilha estão as invocações `empilha(fib_R(3))`, `empilha(fib_R(2))` e `empilha(fib_R(1))`. Invocação só ocupa menos que a anterior, porque há pouca recursão.

3ª Questão (3,0 pontos): Suporte a Orientação a Objetos

3.1 - (1,0 pts) Tipos Abstratos de Dados

Descreva 2 (dois) mecanismos oferecidos por linguagens orientadas a objetos que permitem a construção de tipos abstratos de dados (TADs). Para cada mecanismo, justifique a sua importância para os TADs.

- **RESPOSTA ESPERADA/SUGERIDA**

1. **encapsulamento em classes:** com o encapsulamento de operações (métodos) e atributos na estrutura da classe, permite definir um conjunto determinado de operações para manipulações consistente dos tipos.
2. **proteção aos membros da classe:** com o uso de modificadores de visibilidade como `private` e `public` permite proteger membros da classe (atributos) de modificações diretas que possam fazer com o que o tipo fique em um estado inconsistente.
3. **construtores:** garante que o tipo será sempre corretamente iniciado e utilizado em um estado consistente.

3.2 - (1,0 pts) Classes e Polimorfismo em C++

Considere uma função `disparaTarefa(Tarefa t)`, responsável por executar uma tarefa passada como parâmetro por meio da execução do método `executa()`, conforme indicado no código abaixo. `Tarefa` é uma classe.

```
1 void disparaTarefa(Tarefa t) {  
2     t.executa();  
3 }
```

Um programador precisa descrever dois tipos de tarefa: `ProcessaJobs` e `DisseminaMensagens`.

- a) Declare em C++ as classes `Tarefa`, `ProcessaJobs` e `DisseminaMensagens` de maneira que o objetivo pretendido possa ter atingido. Não é necessário colocar qualquer implementação, mas a função `disparaTarefa(Tarefa t)` não pode ser modificada.

- **RESPOSTA ESPERADA/SUGERIDA**

```
class Tarefa {  
    virtual public void executa();  
}  
  
class ProcessaJobs : public Tarefa{  
    public void executa();  
}  
  
class DisseminaMensagens : public Tarefa{  
    public void executa();  
}
```

// Implementação dos métodos
`public void ProcessaJobs::executa() {`
// implementacao
`}`

`public void DisseminaMensagens::executa() {`
// implementacao
`}`

- b) Descreva um código C++ que crie duas instâncias de objetos `ProcessaJobs` e `DisseminaMensagens` e invoca o método `disparaTarefa(Tarefa t)` para cada uma delas.

Procure deixar o seu código o mais curto possível, escrevendo um *pseudo-código* C++. Pequenos erros de sintaxe não serão considerados.

- **RESPOSTA ESPERADA/SUGERIDA**

```
ProcessaJobs* tarefa1 = new ProcessaJobs()  
DisseminaMensagens* tarefa2 = new Disseminatarefas();  
disparaTarefa(tarefa1)  
disparaTarefa(tarefa2)
```

3.3 - (1,0 pts) Implementação de OO em Linguagens

- a) Refaça a questão anterior para Java e Python. Tente deixar o seu código o mais curto possível.

- **RESPOSTA ESPERADA/SUGERIDA**

```
class Tarefa {  
    public void executa();  
}  
  
class DisseminaMensagens extends Tarefa {  
    public void executa();  
}  
  
class ProcessaJobs extends Tarefa {  
    public void executa();  
}
```

```

}

ProcessaJobs tarefa1 = new ProcessaJobs()
DisseminaMensagens tarefa2 = new Disseminatarefas();
disparaTarefa(tarefa1)
disparaTarefa(tarefa2)

class Tarefa {
    def executa(self) :
        # implementacao
}

class ProcessaJobs( Tarefa ){
    def executa(self):
        # implementacao
}

class DisseminaMensagens ( Tarefa ){
    def executa(self):
        # implementacao
}

tarefa1 = ProcessaJobs()
tarefa2 = DisseminaMensagens()
disparaTarefa(tarefa1)
disparaTarefa(tarefa2)

```

- b) Enumere a diferença entre as implementações Java, Python e C++, considerando os requisitos para que a solução desejada funcione em cada linguagem.
- **C++ vs. Java:** C++ apenas os métodos explicitamente indicados por `virtual` são polimórficos, enquanto em Java eles são por padrão. Em C++, os objetos devem ser criados na heap (e nunca na pilha), utilizando ponteiros. Do contrário, não há polimorfismo nos métodos.
 - **C++/Java vs. Python:** em Python não há exigência sequer que haja uma relação de herança entre as classes, pois ela é dinâmica e utiliza “*tipagem de pato*”. Portanto, basta garantir que as classes `ProcessaJobs` e `DisseminaMensagens` tenham o método `executa` que já é suficiente para a solução ter o comportamento desejado.